

LAPORAN PRAKTIKUM
PEMOGRAMAN ALGORITMA PEMOGRAMAN

PEMOGRAMAN GUI 1

disusun Oleh:

Khaula Lathifa Firdausyi

NIM 2511531007

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Aufan Taufiqurrahman



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	2
1.3 Manfaat Praktikum.....	2
BAB II.....	3
PEMBAHASAN	3
2.1 Praktikum “OperatorAritmatikaGUI”	3
BAB III	12
KESIMPULAN.....	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA	13

KATA PENGANTAR

Laporan praktikum ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan kegiatan praktikum mata kuliah Algoritma Pemograman yang membahas tentang Pemograman GUI di Java. Melalui laporan ini penulis dapat lebih memahami materi praktikum dan dengan penulisan laporan ini dapat melatih ketelitian, keteraturan, serta kemampuan menulis sesuai kaidah akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam pembelajaran selanjutnya.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap segala bentuk kritik dan saran yang bersifat membangun, demi perbaikan kualitas laporan maupun pemahaman penulis di laporan berikutnya.

Padang, 2025

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, khususnya pada pengembangan aplikasi berbasis desktop, penggunaan *Graphical User Interface* (GUI) sangat penting untuk mempermudah interaksi antara pengguna dan sistem. Java, sebagai bahasa pemrograman yang bersifat *platform-independent*, menyediakan *library* *Swing* yang memungkinkan pembuatan antarmuka grafis secara intuitif dan fleksibel. Salah satu penerapan sederhana namun fundamental dari GUI adalah pembuatan kalkulator atau aplikasi perhitungan aritmatika dasar.

Praktikum ini bertujuan untuk mengimplementasikan sebuah aplikasi GUI sederhana menggunakan Java *Swing* yang mampu melakukan operasi aritmatika dasar seperti penjumlahan, pengurangan, perkalian, dan pembagian. Aplikasi ini dirancang dengan tampilan yang *user-friendly*, meliputi komponen-komponen seperti *JTextField* untuk input angka, *JComboBox* untuk memilih operator, dan *JButton* untuk memproses perhitungan. Selain itu, aplikasi juga dilengkapi dengan fitur validasi input dan penanganan error, seperti pemberitahuan jika field kosong atau jika input bukan angka, serta pesan peringatan jika terjadi pembagian dengan nol.

Melalui praktikum ini, mahasiswa diharapkan mampu memahami cara kerja *event-driven programming* dalam Java *Swing* di mana program bereaksi berdasarkan tindakan pengguna, mengelola komponen GUI, serta menerapkan logika pemrograman untuk menangani berbagai kondisi input dan output. Selain itu, praktikum ini juga menjadi dasar bagi pengembangan aplikasi GUI yang lebih kompleks, sekaligus meningkatkan kemampuan dalam merancang antarmuka yang responsif dan mudah digunakan oleh pengguna.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dasar pembuatan antarmuka grafis (*Graphical User Interface/GUI*) dalam pemrograman Java menggunakan *library Swing*.
2. Mampu merancang dan mengimplementasikan aplikasi desktop sederhana yang interaktif, seperti kalkulator aritmatika dasar.
3. Mengetahui dan menerapkan prinsip *event-driven programming*, yaitu bagaimana program merespons tindakan pengguna seperti klik tombol atau pemilihan menu.
4. Melatih kemampuan dalam mengelola komponen GUI seperti *JTextField*, *JComboBox*, *JButton*, dan *JOptionPane* untuk menampilkan pesan.
5. Menerapkan logika pemrograman untuk memproses input pengguna, melakukan operasi aritmatika, serta menangani kesalahan (*error handling*) seperti input tidak valid atau pembagian dengan nol.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Memahami cara membuat antarmuka pengguna/ GUI menggunakan Java Swing.
2. Mengetahui bagaimana membuat aplikasi desktop fungsional seperti kalkulator sederhana.
3. Mengetahui mekanisme *event-driven programming* yang menjadi dasar interaksi pengguna dengan aplikasi GUI.
4. Memahami penggunaan komponen GUI (*JTextField*, *JComboBox*, *JButton*, *JOptionPane*).
5. Meningkatkan kemampuan menulis kode dengan penanganan kesalahan input secara realistis.

BAB II

PEMBAHASAN

2.1 Praktikum “OperatorAritmatikaGUI”

```

package pekan8_2511531007;

import java.awt.EventQueue;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JLabel;
import javax.swing.JOptionPane;

import java.awt.Font;
import java.awt.Color;
import javax.swing.JTextField;
import javax.swing.JComboBox;
import javax.swing.DefaultComboBoxModel;
import javax.swing.SwingConstants;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class OperatorAritmatikaGUI_2511531007 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField txtBil1;
    private JTextField txtBil2;
    private JTextField txtHasil;

    private void pesanPeringatan(String pesan) {
        JOptionPane.showMessageDialog(this, pesan, "Peringatan", JOptionPane.WARNING_MESSAGE);
    }

    private void pesanError(String pesan) {
        JOptionPane.showMessageDialog(this, pesan, "Kesalahan", JOptionPane.ERROR_MESSAGE);
    }

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    OperatorAritmatikaGUI_2511531007 frame = new OperatorAritmatikaGUI_2511531007();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }
}

```

```

// Create the frame.
public OperatorAritmatikaGUI_2511531007() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 300, 300);
    contentPane = new JPanel();
    contentPane.setForeground(new Color(0, 0, 0));
    contentPane.setBackground(new Color(255, 255, 255));
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane);
    contentPane.setLayout(null);

    JLabel lblNewLabel = new JLabel("OPERATOR ARITMATIKA");
    lblNewLabel.setFont(new Font("Tahoma", Font.BOLD, 15));
    lblNewLabel.setBounds(40, 10, 220, 30);
    contentPane.add(lblNewLabel);

    JLabel lblNewLabel_1 = new JLabel("Bilangan 1");
    lblNewLabel_1.setForeground(new Color(0, 0, 0));
    lblNewLabel_1.setBounds(10, 50, 60, 40);
    contentPane.add(lblNewLabel_1);

    JLabel lblNewLabel_1_1 = new JLabel("Bilangan 2");
    lblNewLabel_1_1.setForeground(new Color(0, 0, 0));
    lblNewLabel_1_1.setBounds(10, 70, 60, 40);
    contentPane.add(lblNewLabel_1_1);

    JLabel lblNewLabel_1_2 = new JLabel("Operator");
    lblNewLabel_1_2.setBounds(10, 120, 60, 40);
    contentPane.add(lblNewLabel_1_2);

    JLabel lblNewLabel_1_3 = new JLabel("Hasil");
    lblNewLabel_1_3.setBounds(10, 140, 60, 40);
    contentPane.add(lblNewLabel_1_3);

    txtBil1 = new JTextField();
    txtBil1.setHorizontalAlignment(SwingConstants.CENTER);
    contentPane.add(txtBil1);
    txtBil1.setColumns(10);

    txtBil2 = new JTextField();
    txtBil2.setHorizontalAlignment(SwingConstants.CENTER);
    txtBil2.setColumns(10);
    contentPane.add(txtBil2);
    txtBil2.setBounds(40, 70, 60, 40);
    contentPane.add(txtBil2);

    JComboBox cbOperator = new JComboBox();
    cbOperator.setModel(new DefaultComboBoxModel(new String[] {"+", "-", "*", "/", "%"}));
    cbOperator.setBounds(40, 110, 60, 40);
    contentPane.add(cbOperator);

    txtHasil = new JTextField();
    txtHasil.setEditable(false);
    txtHasil.setBounds(40, 130, 60, 40);
    contentPane.add(txtHasil);
    txtHasil.setColumns(10);
}

```

```

JButton btnNewButton = new JButton("Proses");
btnNewButton.setBackground(new Color(255, 128, 64));
btnNewButton.setForeground(new Color(0, 0, 0));
btnNewButton.addActionListener(new ActionListener() {
    int hasil;

    public void actionPerformed(ActionEvent e) {
        if(txtBil1.getText().trim().isEmpty()) {
            pesanPeringatan("Silahkan input Bilangan 1");
        } else if(txtBil2.getText().trim().isEmpty()) {
            pesanPeringatan("Silahkan input Bilangan 2");
        } else if(txtBil2.getText().trim().startsWith("0")) {
            pesanPeringatan("Bilangan 2 tidak boleh angka 0");
        } else {
            try {
                int a = Integer.parseInt(txtBil1.getText());
                int b = Integer.parseInt(txtBil2.getText());
                int c = cbOperator.getSelectedIndex();
                if(c==0) {hasil = a+b;}
                if(c==1) {hasil = a-b;}
                if(c==2) {hasil = a*b;}
                if(c==3) {hasil = a/b;}
                if(c==4) {hasil = a%b;}
                txtHasil.setText(String.valueOf(hasil));
            } catch (NumberFormatException ex) {
                pesanError("Bilangan 1 dan Bilangan 2 harus Angka");
            }
        }
    }
});
btnNewButton.setBounds(193, 118, 88, 22);
contentPane.add(btnNewButton);
}

```

Gambar 2.1 Kode Program Praktikum “OperatorAritmatikaGUI”

Program ini dibangun menggunakan WindowBuilder dengan Java Swing, yang merupakan alat bantu visual untuk merancang antarmuka pengguna (GUI) dalam pengembangan aplikasi desktop. Aplikasi yang dibuat merupakan kalkulator sederhana yang mampu menjalankan operasi aritmatika dasar, penjumlahan, pengurangan, perkalian, dan pembagian.

Antarmukanya dirancang secara interaktif dengan komponen-komponen seperti kotak input untuk memasukkan bilangan, *combo box* untuk memilih operator, tombol “Proses” untuk menjalankan perhitungan, serta sistem pesan peringatan yang muncul ketika terjadi kesalahan input. Berikut penjelasan mengenai komponen dan konsep utama yang digunakan dalam program tersebut:

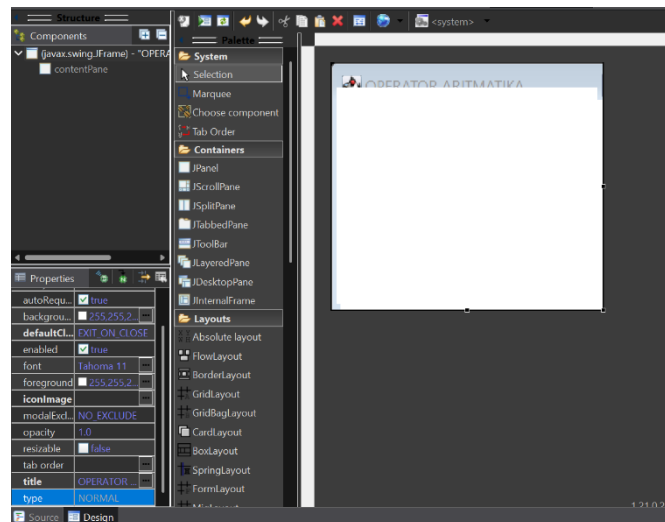
1. Struktur Dasar Program dan Library

Program ini menggunakan beberapa library penting dari Java:

- `javax.swing` → untuk membuat komponen GUI seperti `JFrame`, `JLabel`, `TextField`, `ComboBox`, `Button`, dan `JOptionPane`.
- `java.awt` → untuk mengatur warna (*color*), huruf (*font*), dan (`ActionListener`, `ActionEvent`).
- `java.awt.event` → khusus untuk menangani aksi pengguna seperti klik tombol.
- `javax.swing.border` → untuk memberi batas atau border pada panel.

2. Desain dan Inisialisasi Jendela Utama

Secara visual, jendela aplikasi memiliki:



Gambar 2.1 Visual Jendela Aplikasi

- Judul: “OPERATOR ARITMATIKA”
- Ukuran tetap: lebar 330 piksel, tinggi 300 piksel, diposisikan di koordinat layar (100, 100).
- Latar belakang putih dengan border tipis pada panel utama (contentPane).
- Tidak bisa di-*resize* (setResizable(false)) untuk menjaga tata letak tetap sesuai desain.

Secara kode, konfigurasi ini diatur di bagian *source class*:

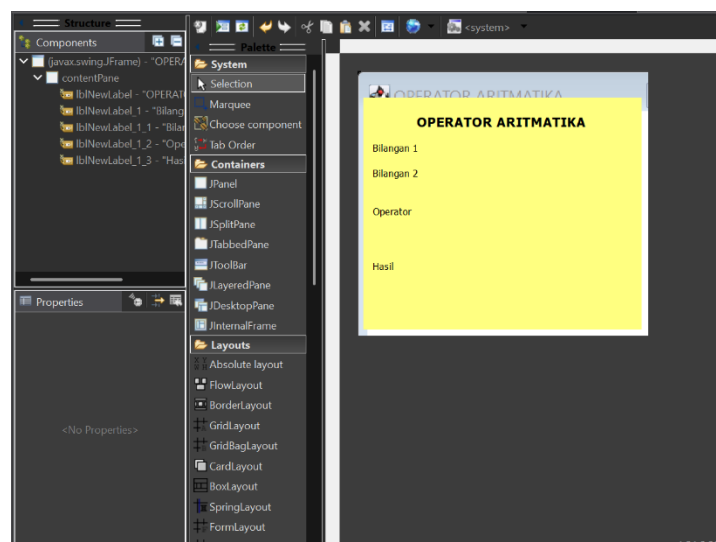
```
setTitle("OPERATOR ARITMATIKA");
setBounds(100, 100, 330, 300);
setResizable(false);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Gambar 2.2 Kode Program Jendela Aplikasi

Panel utama diatur agar memiliki warna latar putih dan border menggunakan `setBorder()` dan `setBackground()`.

3. Komponen Antarmuka dan Implementasinya

A. Label Teks

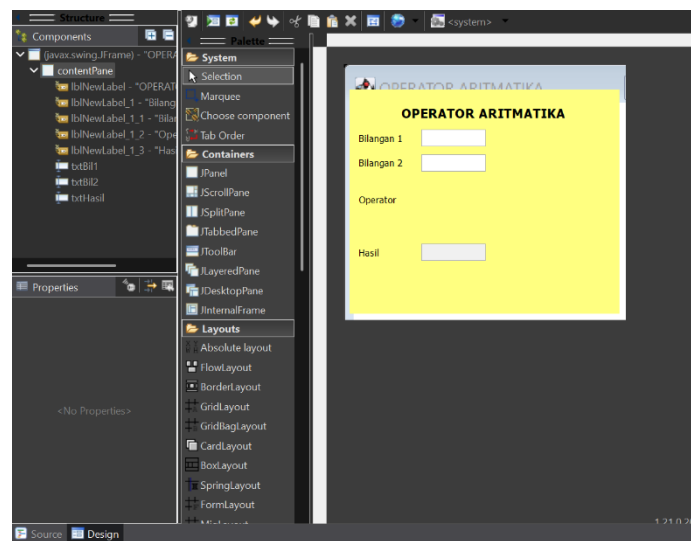


Gambar 2.3 Label Teks

- ❖ Desain: Empat label teks ditempatkan secara vertikal:

- Judul besar di atas: “OPERATOR ARITMATIKA” (*font* Tahoma Bold, ukuran 15).
 - Label instruksi: “Bilangan 1”, “Bilangan 2”, “Operator”, dan “Hasil”.
- ❖ Kode : Masing-masing label dibuat sebagai objek JLabel dan ditempatkan dengan `setBounds(x, y, width, height)`.

B. Kotak Input Angka (JTextField)



Gambar 2.4 Kotak Input Angka

- ❖ Desain :
- Dua kotak input (`txtBill1`, `txtBill2`) untuk memasukkan bilangan.
 - Satu kotak output (`txtHasil`) yang tidak bisa diedit.
 - Semua teks rata tengah untuk konsistensi visual.
- ❖ Kode :

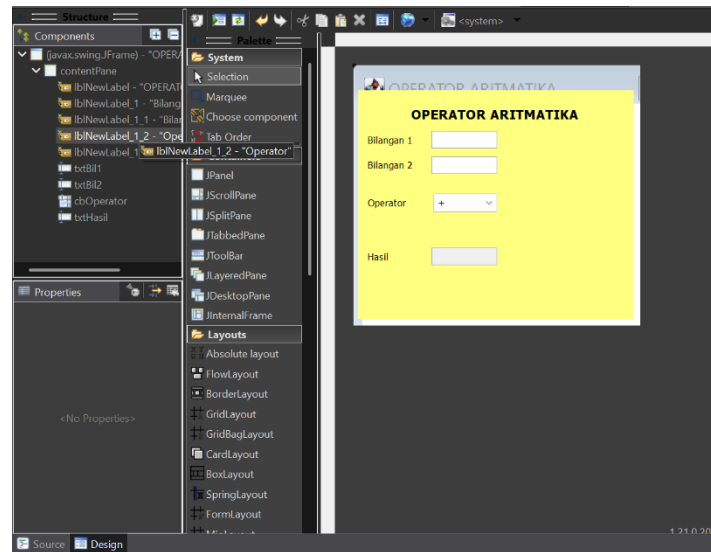
```
txtBill1.setHorizontalAlignment(SwingConstants.CENTER);
txtHasil.setEditable(false);
```

Gambar 2.5 Kode Program Kotak Input Angka

Ukuran dan posisi diatur manual dengan `setBounds()` agar sesuai dengan skema desain visual di WindowBuilder.

C. Combo Box Operator (JComboBox)

- ❖ Desain: Drop-down menu berisi empat operator: +, -, *, /. Ditempatkan di sebelah label “Operator”.



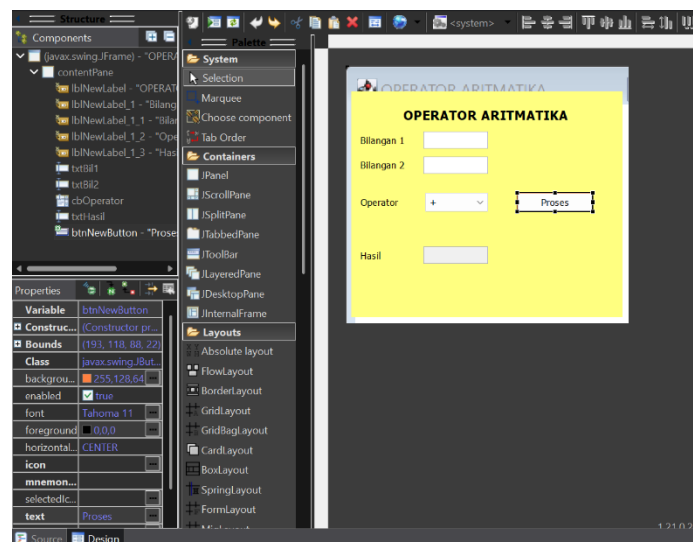
Gambar 2.6 Combo Box Operator

- ❖ Kode: Dibuat menggunakan DefaultComboBoxModel dan diisi secara eksplisit:

```
cbOperator.setModel(new DefaultComboBoxModel(new String[] {"+", "-", "*", "/"}));
```

Gambar 2.7 Kode Program Combo Box Operator

D. Tombol “Proses” (JButton)



Gambar 2.8 Tombol Proses

- ❖ Desain: Tombol berwarna oranye-biru tua (RGB 255,128,64) dengan teks putih, ditempatkan di sisi kanan, sejajar dengan combo box.
- ❖ Kode: Dilengkapi ActionListener untuk menangani klik:

```
btnNewButton.addActionListener(this);
```

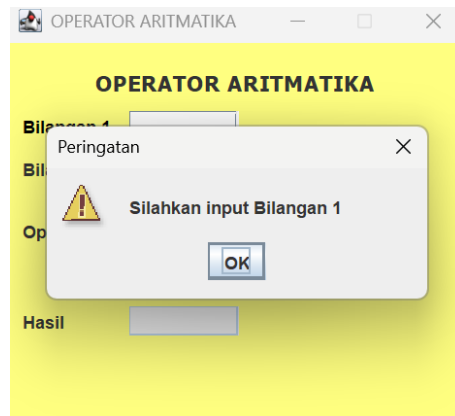
Gambar 2.9 Kode Program Tombol Proses

Saat diklik, metode actionPerformed() dijalankan untuk memproses data.

4. Logika Pemrosesan dan Respons Interaksi Pengguna

Ketika tombol “Proses” diklik, sistem menjalankan rangkaian validasi dan perhitungan berikut:

- ❖ Validasi Input Kosong
 - ❖ Desain Respons: Jika pengguna lupa mengisi, muncul pop-up peringatan.



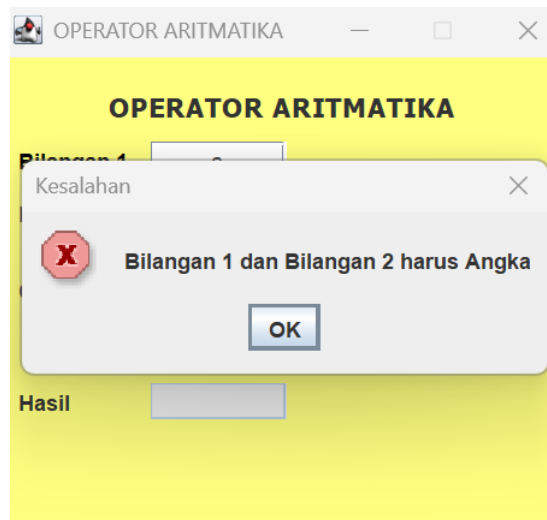
Gambar 2.10 Validasi Input Kosong

- ❖ Kode :

```
if (txtBill1.getText().trim().isEmpty()) {
    pesanPeringatan("Silakan input Bilangan 1");
    return;
}
```

Gambar 2.11 Kode Program Validasi Input Kosong

- ❖ Validasi Format Angka
 - ❖ Desain Respons: Jika pengguna memasukkan huruf atau simbol, muncul pesan error.



Gambar 2.12 Validasi Format Angka

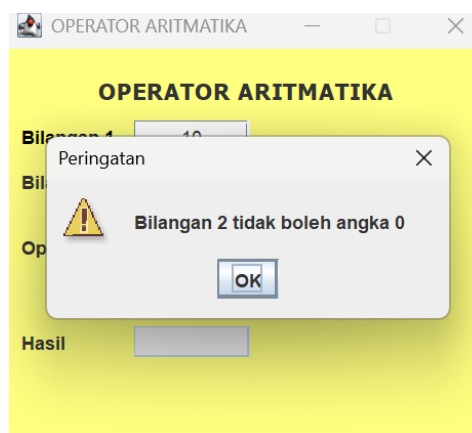
❖ Kode :

```
try {
    int a = Integer.parseInt(txtBill1.getText());
    int b = Integer.parseInt(txtBill2.getText());
} catch (NumberFormatException e) {
    pesanError("Bilangan 1 dan Bilangan 2 harus Angka");
    return;
}
```

Gambar 2.13 Kode Program Validasi Format Angka

✓ Pencegahan Pembagian dengan Nol

❖ Desain Respons: Program tidak crash, melainkan menampilkan peringatan.



Gambar 2.14 Pencegahan Pembagian dengan Nol

❖ Kode:

```
if (cbOperator.getSelectedIndex() == 3 && b == 0) {
    pesanPeringatan("Bilangan 2 tidak boleh angka 0");
    return;
}
```

Gambar 2.15 Kode Program Pencegahan Pembagian dengan Nol

✓ Perhitungan Dinamis

❖ Desain: Hasil langsung muncul di kotak “Hasil”.

Gambar 2.16 Perhitungan Dinamis

❖ Kode :

```
try {
    int a = Integer.parseInt(txtBil1.getText());
    int b = Integer.parseInt(txtBil2.getText());
    int c = cbOperator.getSelectedIndex();
    if(c==0) {hasil = a+b;}
    if(c==1) {hasil = a-b;}
    if(c==2) {hasil = a*b;}
    if(c==3) {hasil = a/b;}
    if(c==4) {hasil = a%b;}
    txtHasil.setText(String.valueOf(hasil));
}
```

Gambar 2.17 Kode program Perhitungan Dinamis

5. Sistem Pesan dan Umpan Balik Pengguna

Program menggunakan dua metode helper untuk menampilkan pesan:

- pesanPeringatan(String msg) → JOptionPane.WARNING_MESSAGE
- pesanError(String msg) → JOptionPane.ERROR_MESSAGE

Ini memastikan bahwa setiap kesalahan input langsung direspons dengan umpan balik visual yang jelas, meningkatkan pengalaman pengguna dan mencegah kebingungan.

6. Titik Masuk Program (main)

Metode `main()` menjalankan GUI dalam *Event Dispatch Thread* (EDT), praktik terbaik dalam pemrograman Swing:

```
EventQueue.invokeLater(new Runnable() {  
    public void run() {  
        try {  
            OperatorAritmatikaGUI_2511531007 frame = new OperatorAritmatikaGUI_2511531007();  
            frame.setVisible(true);  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
});
```

Gambar 2.18 Metode Main

BAB III

KESIMPULAN

3.1 Kesimpulan

Praktikum ini berhasil mengimplementasikan sebuah aplikasi desktop sederhana berbasis *Graphical User Interface* (GUI) menggunakan Java Swing dan alat bantu WindowBuilder. Aplikasi yang dibuat mampu melakukan operasi aritmatika dasar, penjumlahan, pengurangan, perkalian, dan pembagian dengan antarmuka yang interaktif dan responsif terhadap input pengguna.

Melalui praktikum ini, dapat dipahami pentingnya integrasi antara desain antarmuka dan logika pemrograman: komponen GUI seperti `JTextField`, `JComboBox`, dan `JButton` tidak hanya berfungsi sebagai elemen visual, tetapi juga harus dihubungkan dengan mekanisme *event handling* dan validasi data agar aplikasi dapat berjalan dengan andal. Selain itu, penanganan kesalahan seperti input kosong, bukan angka, atau pembagian dengan nol menunjukkan prinsip dasar pembuatan perangkat lunak yang *user-friendly* dan tahan terhadap kesalahan umum.

DAFTAR PUSTAKA

[1] Oracle, “The Java™ Tutorials: Creating a GUI With JFC/Swing,” Oracle Documentation, 2025. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/>

[Accessed: Nov. 21, 2025].

[2] Eclipse Foundation, “WindowBuilder User Guide,” Eclipse IDE Documentation, 2025. [Online]. Available: <https://www.eclipse.org/windowbuilder/>

[Accessed: Nov. 21, 2025].

[3] H. M. Deitel and P. J. Deitel, Java How to Program, Early Objects, 11th ed. Hoboken, NJ, USA: Pearson, 2018.

[4] A. Shalahuddin and R. A. Hidayat, Pengantar Pemrograman Java: Dari Dasar hingga GUI dengan Swing, 2nd ed. Bandung, Indonesia: Informatika, 2021.

[5] S. W. Surya, Pemrograman Berorientasi Objek dengan Java, Yogyakarta, Indonesia: Graha Ilmu, 2020.